

El Kit de Herramientas de Python

Desde la instalación hasta las bibliotecas, descubre el ecosistema esencial para desarrollar proyectos en Python de forma organizada y eficiente.



Primeros Pasos para Aprender Python

¿Por qué un kit de herramientas?

Al comenzar con Python, la cantidad de herramientas disponibles puede abrumar. Usar un **conjunto básico y bien definido** simplifica el proceso y permite enfocarse en aprender a programar.

El kit básico te permite:

- Escribir y organizar código
- Gestionar proyectos de forma independiente
- Instalar bibliotecas externas
- Desarrollar aplicaciones completas

PYTHON PROGRAMMING for BEGINNERS



The Absolute Complete Beginner's Guide to
Learn & Apply Python Programming Language
Using Best Practices and Advanced Features

JAMES HERRON

Configuración del Entorno de Python

Configurar correctamente el entorno es el **primer paso fundamental** antes de comenzar cualquier proyecto. Sin esta base, el resto de las herramientas no podrán funcionar adecuadamente.



IDE: Entorno de Desarrollo Integrado

Un IDE es un programa diseñado específicamente para escribir y gestionar código. Puede compararse con un procesador de texto, pero con herramientas adicionales para programar.

```
def __init__(self):
    self.file = None
    self.fingerprints = set()
    self.logdups = True
    self.debug = debug
    self.logger = logging.getLogger(__name__)
    if path:
        self.file = open(os.path.join(path, 'log.txt'), 'w')
        self.file.seek(0)
        self.fingerprints.update(self.request_fingerprint(request))

    @classmethod
    def from_settings(cls, settings):
        debug = settings.getbool('SUPERFUTUR_PANORAMA')
        return cls(job_dir(settings), debug)

    def request_seen(self, request):
        fp = self.request_fingerprint(request)
        if fp in self.fingerprints:
            return True
        self.fingerprints.add(fp)
        if self.file:
            self.file.write(fp + os.linesep)

    def request_fingerprint(self, request):
        return request_fingerprint(request)
```

¿Qué ofrece un IDE?



Resaltado de sintaxis.

Colorea el código para hacerlo más fácil de leer y comprender.



Autocompletado.

Sugiere funciones, variables y fragmentos de código a medida que escribes.



Depuración.

Detecta y corrige errores al analizar el código paso a paso.



Ejecución directa.

Permite ejecutar programas sin salir del entorno de desarrollo.



IAs Integradas.

Accesos a Modelos de IAs para planificar, codificar, entender y preguntar dudas .

Entornos Virtuales

El problema

Al desarrollar varios proyectos en

Python, pueden surgir

conflictos entre las bibliotecas

utilizadas en cada uno.

Diferentes proyectos pueden
requerir versiones distintas de una
misma biblioteca.

La solución: entornos virtuales

Un entorno virtual actúa como un

contenedor independiente

para cada proyecto,

permitiendo instalar bibliotecas

distintas sin conflictos y

manteniendo los proyectos

organizados y seguros.



pip: Gestor de Paquetes de Python

[Ayuda](#)[Docs](#)[Patrocinadores](#)[Acceder](#)[Registrarse](#)

Encuentre, instale y publique paquetes de Python con el Índice de paquetes de Python



O bien, [explore los proyectos](#)

760.212 proyectos

8.214.662 versiones

17.651.561 archivos

1.022.452 usuarios

Cómo Encajan Todas las Herramientas

1

Python

Ejecuta los programas

2

IDE

Escribe y desarrolla código

3

Entornos Virtuales

Organiza proyectos

4

Bibliotecas + pip

Añade funcionalidades

Juntas, estas herramientas forman el **kit esencial** para cualquier desarrollador de Python. Con práctica, se convierten en la base para crear aplicaciones profesionales y contribuir a proyectos de código abierto.

IDE EN PYTHON

IDE Populares para Python

La elección del IDE depende del **nivel de experiencia**, el tipo de proyecto y las preferencias personales. Tres opciones destacan en el ecosistema Python.

e.ipynb M X (R) vscode-notebook.rst 7, U

dev > blog > code > 2021-11 > VSCodeArticle.ipynb > M↓A

larkdown | ▶ Run All ≡ Clear Outputs of All Cells ↺ R

de Notebooks

```
import pandas as pd
import numpy as np
```

t Data

```
url = 'https://github.com/chris1610/pbpython/'
large file for timing test
url = 'https://data.iowa.gov/api/views/m3tr-'
df = pd.read_excel(url)
```

Data

Jupyter Notebook

Ideal para:

- Ciencia de datos
- Investigación
- Análisis de datos
- Educación

Características principales

Celdas interactivas que se ejecutan de forma independiente

Resultados visuales con gráficos y tablas integrados

Soporte Markdown para combinar código, texto y ecuaciones

Visual Studio Code y PyCharm

Visual Studio Code

IDE versátil y muy popular. Incluye **autocompletado inteligente**, depuración avanzada, integración con Git y un marketplace con miles de extensiones para desarrollo web, ciencia de datos y más.

PyCharm

Diseñado específicamente para **desarrollo profesional en Python**. Ofrece depuración avanzada, refactorización de código, pruebas automatizadas y gestión de proyectos complejos.

Jupyter

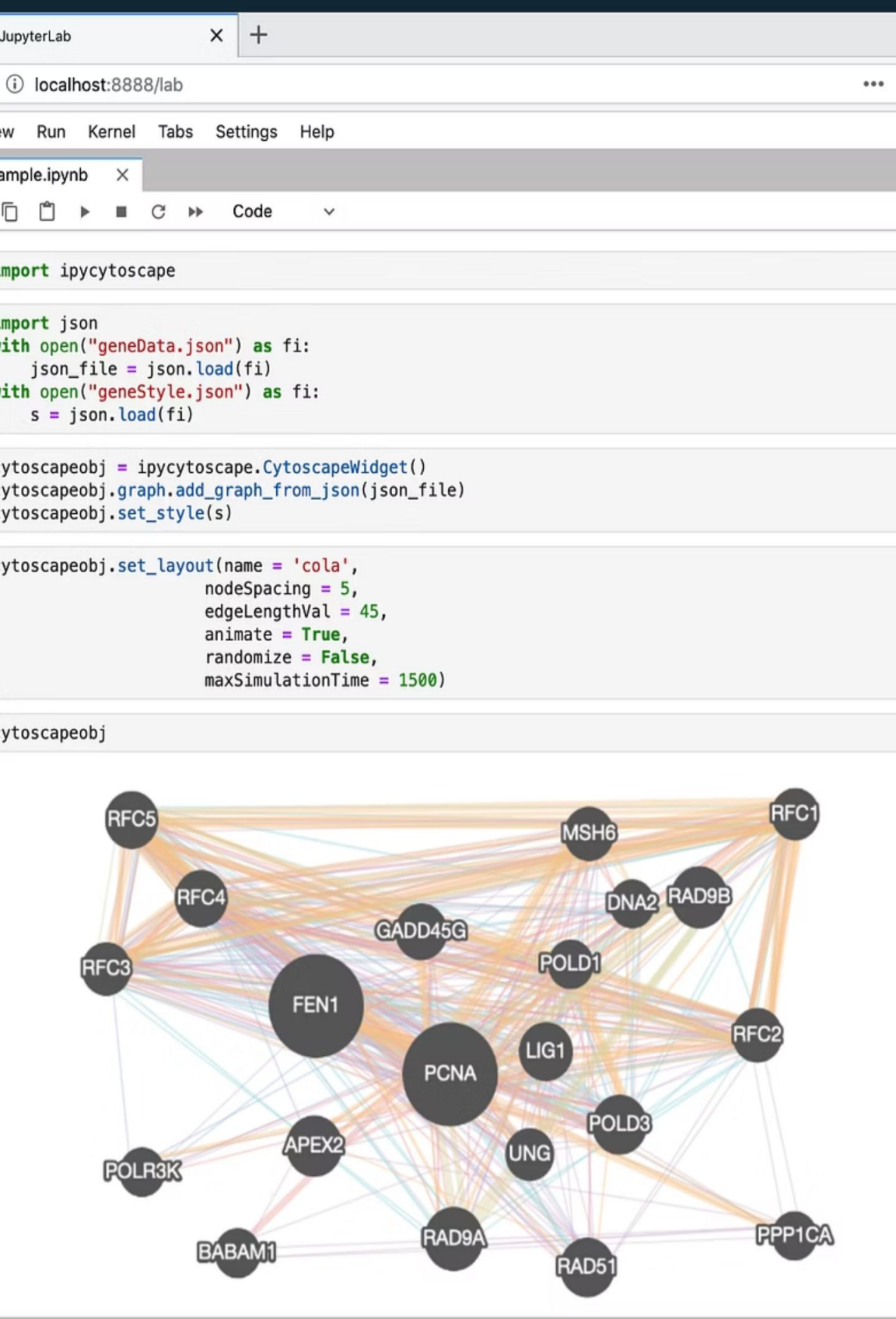
Análisis y experimentación

VS Code

Proyectos variados

PyCharm

Desarrollo profesional



JUPYTER NOTEBOOK

¿Qué es Jupyter Notebook?

Jupyter Notebook es una plataforma interactiva que permite escribir y ejecutar código mientras se documenta el proceso en el mismo archivo.

Puede imaginarse como un **laboratorio digital** donde es posible escribir código, ejecutar experimentos, documentar resultados, crear visualizaciones y compartir el trabajo completo.

Estructura y Ventajas de Jupyter Notebook

Dos tipos de celdas

Celdas de Código

Ejecutan Python y muestran resultados inmediatamente debajo.

Celdas Markdown

Permiten agregar títulos, listas, ecuaciones e imágenes para documentar el análisis.

Ventajas clave

Interactividad: ejecuta bloques individuales sin correr todo el programa

Análisis de datos: integración con Pandas, NumPy y Matplotlib

Documentación integrada: combina análisis técnico y explicación

Reproducibilidad: comparte flujos de trabajo completos

Interfaz y Funciones Avanzadas de Jupyter

O1

Barra de Herramientas

Guardar, crear celdas, ejecutar y reiniciar el kernel desde la parte superior.

O2

Selector de Tipo de Celda

Cambia entre modo **Code** y **Markdown** según el contenido a crear.

O3

Comandos Mágicos (%)

Permiten mostrar gráficos, medir tiempos de ejecución y ejecutar comandos del sistema.

O4

Extensiones

Añaden funciones como plegado de código, inspector de variables e integración con Git.

📌 El kernel es el motor que ejecuta el código.

Conclusión: Tu Base como Desarrollador Python

Ecosistema Completo

Python, IDE, entornos virtuales, bibliotecas y pip forman una base sólida para cualquier proyecto.

Productividad Real

Los IDE y Jupyter Notebook permiten escribir, depurar y documentar código de forma eficiente.

Crecimiento Profesional

Con práctica, estas herramientas se convierten en la base para aplicaciones profesionales y proyectos de código abierto.



Sintaxis básica de Python

La **sintaxis** es el conjunto de reglas que determinan cómo se escriben las instrucciones en Python. Al igual que un idioma tiene gramática, Python tiene reglas que permiten organizar palabras y símbolos para que la computadora entienda exactamente qué debe hacer.

Indentación

Estructura visual del código

Sentencias

Instrucciones para la máquina

Variables

Contenedores de datos

Comentarios

Notas para los programadores

Comprender estos cuatro elementos es esencial para escribir programas correctos y fáciles de leer. Un primer ejemplo clásico es el programa **Hello World**, que simplemente muestra un mensaje en pantalla.

¿Qué es la sintaxis?

En programación, la **sintaxis** se refiere al conjunto de reglas que indican cómo deben organizarse las instrucciones en un lenguaje de programación. Es, en esencia, la gramática del código.

Como en un idioma humano...

El español tiene reglas para construir oraciones correctas: sujeto, verbo, predicado.

Sin esas reglas, la comunicación se rompe.

En Python, la sintaxis define:

- Cómo escribir el código correctamente
- Cómo estructurar las instrucciones
- Cómo comunicar ideas a la computadora

Gracias a la sintaxis, las ideas del programador se convierten en **acciones que la máquina puede ejecutar**.

Sin una sintaxis correcta, Python no puede interpretar el código y generará un error.
Aprender la sintaxis es el primer paso para hablar el lenguaje de las computadoras.

Elementos fundamentales de la sintaxis en Python

Python organiza su código a través de cuatro pilares fundamentales. Cada uno cumple una función específica y todos trabajan en conjunto para crear programas claros y funcionales.

```
class Flight:
    """A flight with a particular aircraft."""
    def __init__(self, number, aircraft):
        if not number[2].isdigit():
            raise ValueError("No numeric code in '{}'".format(number))
        if not number[2].isupper():
            raise ValueError("Invalid airline code '{}'".format(number))
        if not (number[2].isdigit() and int(number[2]) < 1000):
            raise ValueError("Invalid route number '{}'".format(number))
        self._number = number
        self._aircraft = aircraft
        self._seating = None
        rows, seats = self._aircraft.seating_plan()
        self._seating = [None] * [ (letter,row) for letter in rows ]

    def _passenger_seats(self):
        """An iterator series of passenger seating allocations."""
        row_numbers, seat_letters = self._aircraft.seating_plan()
        for row in row_numbers:
            for letter in seat_letters:
                passenger = self._seating[row][letter]
                if passenger is not None:
                    yield (passenger, "%3s" % letter)
```

Indentación

Define los bloques de código mediante espacios. Es obligatoria en Python, a diferencia de otros lenguajes que usan llaves `{ }`.

```
/ find _
-
./http:
./http:
./http://scripting.tips
./http://scripting.tips/complete-cli-guide.html
./http://scripting.tips/index.html

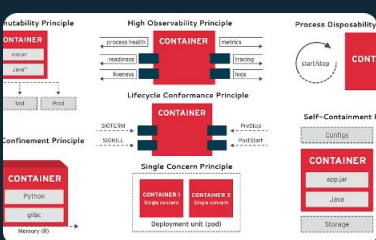
/demo
> find _ -type f
./http://scripting.tips/complete-cli-guide.html
./http://scripting.tips/index.html

/demo
> find _ -type d
-
./http:
./http:
./http://scripting.tips

/demo
```

Sentencias

Son las instrucciones individuales que le damos a la computadora. Cada línea suele representar una acción concreta.



Variables

Contenedores que almacenan datos con un nombre descriptivo, permitiendo reutilizar y actualizar información fácilmente.



Comentarios

Notas escritas en el código que Python ignora al ejecutar. Sirven para explicar la lógica y mejorar la comprensión.

Los cuatro pilares en detalle

Indentación

Agregar espacios al inicio de una línea define bloques de código. En Python, la **indentación no es opcional**: si está mal alineada, el programa genera un error. Cada nivel anidado representa un bloque interior.

```
if edad >= 18:
    print("Eres mayor de edad")
else:
    print("Eres menor de edad")
```

Variables

Contenedores que almacenan datos de distintos tipos (números, texto, listas). Pueden actualizarse durante la ejecución, haciendo el código más flexible y adaptable.

```
edad = 25
nombre = "Ana"
```

Sentencias

Una sentencia es una instrucción que realiza una acción: asignar valores, hacer cálculos, llamar funciones o controlar el flujo. Forman la lógica completa del programa.

```
resultado = 10 + 5
```

Permitido pero No Recomendado (No Pitonico) el uso de ;

```
pre = 10 ; cant = 2 ; print ("resulatado: ", pre * cant)
```

Comentarios

Líneas precedidas por # que Python ignora completamente. Sirven para explicar la lógica, dejar recordatorios y documentar decisiones, especialmente en proyectos colaborativos.

```
# Este programa calcula la edad del usuario
edad = 25
```


Programa Hello World en Python

El programa **Hello World** es el primer paso tradicional al aprender cualquier lenguaje de programación. Su objetivo es simple: mostrar un mensaje en pantalla. Aquí está el proceso completo:

Crear archivo

Escribir código

Guardar y ejecutar

El código completo

```
print("Hello, World!")
```

Una sola línea es suficiente para tu primer programa en Python.

Dos formas de ejecutarlo

Desde un IDE: usando el botón Run o Ejecutar

Desde la terminal: escribiendo `python hello.py`

El resultado en pantalla será: `Hello, World!`

¡EXAMEN HOY!



¡NO OLVIDES IR!